

Reporte Técnico: Implementación de Firma Digital ECDSA desde Cero

Proyecto de Matemáticas Discretas 2 — Ingeniería de Sistemas y Computación

Resumen

Este proyecto implementa un sistema de firma digital basado en el algoritmo **ECDSA** (*Elliptic Curve Digital Signature Algorithm*) construido enteramente desde cero en Python, sin recurrir a librerías criptográficas externas, con una interfaz grafica hecha con la herramienta claud code empleando streamlit para evitar el uso de codigo HTML o JSS. El objetivo es doble: por un lado, producir una herramienta funcional capaz de generar claves, firmar mensajes y verificar firmas; por otro, servir como demostración didáctica donde cada operación es trazable y se justifica con los contenidos del curso —teoría de grupos y teoría de números—. El sistema opera sobre el grupo abeliano de puntos de una curva elíptica definida sobre un campo finito, y su seguridad descansa en el problema del logaritmo discreto elíptico. El proyecto incluye además una demostración del ataque por reutilización de nonce, que evidencia cómo un error de implementación compromete por completo la clave privada.

Todo el codigo respectivo pueden encontrarlo en el repositorio de Github:

https://github.com/jaacmameciji23/firma_digital_ECDSA

y la pagina web para ver el resultado final es:

<https://jaacmameciji23-firma-digital-ecdsa-app-itfeze.streamlit.app/>

1. Introducción y objetivos

Una firma digital es un mecanismo que permite verificar dos propiedades de un mensaje: su **autenticidad** (fue creado por quien dice haberlo creado) y su **integridad** (no fue alterado tras la firma). A diferencia del cifrado, una firma no oculta el contenido del mensaje; lo acompaña con una prueba matemática que solo el poseedor de la clave privada pudo generar, pero que cualquiera puede verificar con la clave pública.

Los objetivos concretos del proyecto son:

- Implementar la aritmética del grupo de una curva elíptica sobre un campo finito, verificando los axiomas de grupo.

- Construir el protocolo ECDSA completo: generación de claves, firma y verificación.
- Demostrar el ataque por reutilización de nonce como caso de estudio de seguridad.
- Garantizar que cada paso sea legible y esté conectado con un concepto teórico del curso.

El proyecto integra los dos bloques temáticos de la asignatura: la **teoría de grupos y estructuras algebraicas** (el grupo de puntos de la curva, generadores, orden, isomorfismos) y la **teoría de números y criptografía** (aritmética modular, algoritmo de Euclides extendido, construcción de un primitivo criptográfico).

2. Fundamento matemático

2.1 La curva elíptica sobre un campo finito

Una curva elíptica sobre el campo finito $\mathbb{Z}_p$ (con p primo) es el conjunto de puntos (x, y) que satisfacen la ecuación:

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

junto con un punto especial llamado **punto al infinito**, denotado O , que actúa como elemento neutro. Para que la curva sea no singular (condición necesaria para que la operación de grupo esté bien definida) se requiere que el discriminante cumpla $4a^3 + 27b^2 \not\equiv 0 \pmod{p}$.

En este proyecto se emplea la curva:

$$y^2 \equiv x^3 + 4x + 7 \pmod{11}$$

es decir, $a = 4$, $b = 7$, $p = 11$.

2.2 El grupo abeliano de puntos

El conjunto de puntos de la curva, con la operación de suma de puntos, forma un **grupo abeliano**. Esto significa que cumple cinco propiedades:

Propiedad	Descripción	En nuestra curva
Cierre	La suma de dos puntos es otro punto de la curva	Toda suma produce uno de los 13 puntos
Asociatividad	$(P+Q)+R = P+(Q+R)$	Heredada de la estructura algebraica

Propiedad	Descripción	En nuestra curva
Elemento neutro	Existe O tal que $P+O = P$	El punto al infinito
Inverso	Todo P tiene un -P tal que $P+(-P) = O$	El inverso de (x,y) es (x, p-y)
Conmutatividad	$P+Q = Q+P$	La suma de puntos es simétrica

Para nuestra curva, el grupo tiene exactamente **13 elementos** (12 puntos afines más O). Dado que 13 es primo, el grupo es cíclico y todo elemento distinto de O es un generador.

2.3 El conjunto completo de puntos

Evaluando la ecuación para cada $x \in \{0, \dots, 10\}$ y buscando los valores de y correspondientes, se obtiene el grupo completo:

$$E(Z_{11}) = \{ (1,1), (1,10), (2,1), (2,10), (5,3), (5,8), \\ (6,4), (6,7), (7,2), (7,9), (8,1), (8,10), O \}$$

El proyecto toma como generador $G = (1, 1)$. Las multiplicaciones escalares sucesivas de G recorren todo el grupo:

$$1 \cdot G = (1, 1) \quad 8 \cdot G = (6, 7)$$

$$2 \cdot G = (2, 1) \quad 9 \cdot G = (5, 3)$$

$$3 \cdot G = (8, 10) \quad 10 \cdot G = (8, 1)$$

$$4 \cdot G = (5, 8) \quad 11 \cdot G = (2, 10)$$

$$5 \cdot G = (6, 4) \quad 12 \cdot G = (1, 10)$$

$$6 \cdot G = (7, 2) \quad 13 \cdot G = O$$

$$7 \cdot G = (7, 9)$$

Se observa que $13 \cdot G = O$, confirmando que el orden de G es $n = 13$.

3. Algoritmos implementados

El sistema se organiza en cuatro módulos: `grupos.py` (aritmética del grupo), `ecdsa.py` (protocolo), `ataque.py` (caso de estudio de seguridad) y `demo.py` (interfaz). A continuación se describen los algoritmos centrales.

3.1 Algoritmo de Euclides extendido

Es la pieza aritmética fundamental del proyecto. Su función es calcular **inversos modulares**, necesarios en la división modular que aparece en la suma de puntos, en la firma y en la verificación. Dado que en aritmética modular no existe la división directa, se recurre a la identidad de Bézout: para dos enteros a y n con $\text{mcd}(a, n) = 1$, existen x , y tales que $a \cdot x + n \cdot y = 1$, de donde $a \cdot x \equiv 1 \pmod{n}$, es decir, x es el inverso de a .

```
def euclides_extendido(a, b, verbose=True):
    """
    Algoritmo de Euclides Extendido.
    Retorna (mcd, x, y) tal que  $a \cdot x + b \cdot y = \text{mcd}(a, b)$ .
    """
    old_r, r = a, b
    old_s, s = 1, 0
    old_t, t = 0, 1

    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t

    return old_r, old_s, old_t
```

El inverso modular se obtiene directamente a partir de este resultado:

```
def inverso_modular(a, p, verbose=True):
    a_mod = a % p
    mcd, x, _ = euclides_extendido(a_mod, p, verbose)
    if mcd != 1:
        raise ValueError(f"No existe inverso de {a} mod {p}")
    return x % p
```

Ejemplo: para calcular $5^{-1} \bmod 13$, el algoritmo resuelve $5 \cdot x + 13 \cdot y = 1$, encuentra $x = -5$, y como $-5 \equiv 8 \pmod{13}$, concluye que $5^{-1} = 8$. Se verifica: $5 \cdot 8 = 40 \equiv 1 \pmod{13}$.

3.2 Suma de puntos

Es la operación del grupo. Distingue tres casos: cuando uno de los puntos es O (elemento neutro), cuando $P = Q$ (duplicación, que usa la recta tangente) y cuando $P \neq Q$ (suma estándar). En ambos casos no triviales se calcula una pendiente λ y a partir de ella las coordenadas del resultado:

$$\text{Caso } P \neq Q: \quad \lambda = (y_2 - y_1) \cdot (x_2 - x_1)^{-1} \bmod p$$

$$\text{Caso } P = Q: \quad \lambda = (3x_1^2 + a) \cdot (2y_1)^{-1} \bmod p$$

$$x_3 = \lambda^2 - x_1 - x_2 \bmod p$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \bmod p$$

El cálculo de λ requiere un inverso modular, lo que conecta esta operación directamente con el algoritmo de Euclides extendido.

```

def suma_puntos(P, Q, a, p, verbose=True):
    if P is None:
        return Q          # 0 + Q = Q
    if Q is None:
        return P          # P + 0 = P

    x1, y1 = P
    x2, y2 = Q

    # Puntos inversos: mismo x, ordenadas opuestas -> resultado 0
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:             # Duplicación: recta tangente
        num = (3 * x1 * x1 + a) % p
        den = (2 * y1) % p
    else:                  # Suma estándar: recta secante
        num = (y2 - y1) % p
        den = (x2 - x1) % p

    lam = (num * inverso_modular(den, p, verbose)) % p
    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return (x3, y3)

```

3.3 Multiplicación escalar (double-and-add)

Calcular $k \cdot G$ sumando G consigo mismo k veces es ineficiente. El algoritmo **double-and-add** reduce el costo de $O(k)$ a $O(\log k)$ representando k en binario: por cada bit se duplica el acumulador, y si el bit es 1 se suma además G .

```
def multiplicacion_escalar(k, G, a, p, verbose=True):
    """
    Calcula k·G usando double-and-add (MSB first).
    Complejidad: O(log k) sumas de puntos.
    """
    bits = bin(k)[2:]
    R = None # punto en el infinito 0

    for bit in bits:
        if R is not None:
            R = suma_puntos(R, R, a, p, verbose) # doblar
        if bit == '1':
            R = G if R is None else suma_puntos(R, G, a, p, verbose) # sumar
    return R
```

Ejemplo: para $k = 8 = 1000_2$, el algoritmo parte de G y realiza tres duplicaciones consecutivas: $G \rightarrow (2,1) \rightarrow (5,8) \rightarrow (6,7)$. El resultado $8 \cdot G = (6,7)$ se obtiene con 3 operaciones en lugar de 8.

4. El protocolo ECDSA

4.1 Generación de claves

La clave privada d es un entero aleatorio en el rango $[1, n-1]$. La clave pública Q es el punto resultante de la multiplicación escalar $Q = d \cdot G$.

```
def generar_claves(curva=None, verbose=True):
    a, p, G, n = curva['a'], curva['p'], curva['G'], curva['n']
    d = random.randint(1, n - 1) # clave privada
    Q = multiplicacion_escalar(d, G, a, p, verbose) # clave pública
    return d, Q
```

La seguridad se fundamenta en la asimetría de esta operación: calcular Q a partir de d es eficiente, pero recuperar d a partir de Q es el **problema del logaritmo discreto elíptico (ECDLP)**, para el cual no se conoce algoritmo eficiente en curvas adecuadamente elegidas.

Ejemplo: con $d = 8$ se obtiene $Q = 8 \cdot G = (6, 7)$.

4.2 Firma

Para firmar un mensaje se ejecutan cinco pasos. El algoritmo produce un par (r, s) que constituye la firma.

1. $h = \text{hash}(\text{mensaje}) \bmod n$
2. $k = \text{nonce aleatorio en } [1, n-1]$
3. $R = k \cdot G$
4. $r = R.x \bmod n$ (si $r = 0$, elegir otro k)
5. $s = k^{-1} \cdot (h + d \cdot r) \bmod n$ (si $s = 0$, elegir otro k)

```
def firmar(mensaje, d, curva=None, k=None, verbose=True):
    a, p, G, n = curva['a'], curva['p'], curva['G'], curva['n']

    h = hash_mensaje(mensaje, n, verbose)          # paso 1
    if k is None:
        k = random.randint(1, n - 1)              # paso 2
    R = multiplicacion_escalar(k, G, a, p, verbose) # paso 3
    r = R[0] % n                                    # paso 4

    k_inv = inverso_modular(k, n, verbose)          # paso 5 (Euclides)
    s = (k_inv * (h + d * r)) % n
    return r, s
```

El paso 5 requiere el inverso modular de k , calculado con el algoritmo de Euclides extendido. Aquí aparece de nuevo la conexión con la teoría de números.

Ejemplo: firmando el mensaje "Hola" con $d = 8$ y $k = 5$:

$$h = 11$$

$$R = 5 \cdot G = (6, 4)$$

$$r = 6$$

$$k^{-1} = 5^{-1} \bmod 13 = 8$$

$$s = 8 \cdot (11 + 8 \cdot 6) \bmod 13 = 8 \cdot 7 \bmod 13 = 4$$

Firma = (6, 4)

4.3 Verificación

El verificador conoce únicamente la clave pública Q , el mensaje y la firma (r, s) . No conoce ni d ni k . El procedimiento reconstruye un punto X y comprueba que su coordenada x coincide con r .

1. $h = \text{hash}(\text{mensaje}) \bmod n$
2. $w = s^{-1} \bmod n$
3. $u_1 = h \cdot w \bmod n$
4. $u_2 = r \cdot w \bmod n$
5. $X = u_1 \cdot G + u_2 \cdot Q$
6. Válida $\Leftrightarrow X.x \bmod n = r$

```
def verificar(mensaje, firma, Q, curva=None, verbose=True):
    a, p, G, n = curva['a'], curva['p'], curva['G'], curva['n']
    r, s = firma

    h = hash_mensaje(mensaje, n, verbose)
    w = inverso_modular(s, n, verbose)    # Euclides
    u1 = (h * w) % n
    u2 = (r * w) % n

    X1 = multiplicacion_escalar(u1, G, a, p, verbose)
    X2 = multiplicacion_escalar(u2, Q, a, p, verbose)
    X = suma_puntos(X1, X2, a, p, verbose)

    return X is not None and X[0] % n == r
```

Los valores u_1 y u_2 son pesos auxiliares que, combinados en $X = u_1 \cdot G + u_2 \cdot Q$, reconstruyen el punto R generado durante la firma.

4.4 Correctitud: por qué funciona

La verificación es válida gracias a las propiedades del grupo abeliano. Sustituyendo $Q = d \cdot G$ y desarrollando:

$$X = u_1 \cdot G + u_2 \cdot Q$$

$$= (h \cdot w) \cdot G + (r \cdot w) \cdot (d \cdot G)$$

$$= w \cdot (h + r \cdot d) \cdot G \quad [\text{distributividad y conmutatividad}]$$

$$= s^{-1} \cdot (h + r \cdot d) \cdot G \quad [w = s^{-1}]$$

$$= k \cdot G \quad [\text{pues } s = k^{-1} \cdot (h + r \cdot d) \Rightarrow s^{-1} \cdot (h + r \cdot d) = k]$$

$$= R$$

Por tanto $X \cdot x = R \cdot x = r$ y la firma se acepta. La cadena de igualdades depende esencialmente de que la operación sea **asociativa y conmutativa**: sin estas propiedades no sería posible factorizar ni reordenar los términos.

4.5 Detección de alteraciones

Si un atacante modifica el mensaje, su hash cambia y la reconstrucción de X ya no coincide con R . Por ejemplo, al alterar "Hola" a "Hola[modificado]", el hash pasa de 11 a 12, y la verificación produce $X = (2, 1)$, cuya coordenada x es $2 \neq 6 = r$, de modo que la firma se rechaza automáticamente. Esto garantiza la integridad del mensaje.

5. Ejemplo completo paso a paso

Esta sección recorre el ciclo completo del sistema con valores concretos, desde la configuración inicial hasta la detección de un mensaje alterado. Todos los números provienen de la ejecución real del código. El caso de estudio emplea la clave privada $d = 8$, el mensaje "Hola" y el nonce $k = 5$, este número es aleatorio pero para este ejemplo se fija el valor.

Visión general del flujo

FASE 0 Configuración $\rightarrow$ curva, generador G , orden n

FASE 1 Generación claves $\rightarrow d$ (privada) $\rightarrow Q = d \cdot G$ (pública)

FASE 2 Firma $\rightarrow h \rightarrow k \rightarrow R = k \cdot G \rightarrow r \rightarrow s \Rightarrow (r, s)$

FASE 3 Verificación $\rightarrow w \rightarrow u_1, u_2 \rightarrow X = u_1 \cdot G + u_2 \cdot Q \Rightarrow \text{¿}X.x = r\text{?}$

FASE 4 Alteración $\rightarrow \text{hash distinto} \rightarrow X \text{ distinto} \Rightarrow \text{rechazo}$

Fase 0 — Configuración del sistema

Los parámetros públicos son conocidos por todos los participantes y no cambian:

Parámetro	Valor
Curva	$y^2 \equiv x^3 + 4x + 7 \pmod{11}$
Módulo del campo	$p = 11$
Generador	$G = (1, 1)$
Orden de G	$n = 13$ (primo)

Pueden observar que se manejan **dos módulos distintos** con roles diferentes: $p = 11$ gobierna las coordenadas de los puntos (toda la aritmética dentro de `suma_puntos`), mientras que $n = 13$ gobierna los escalares (hash, nonce, r , s y todos los cálculos del protocolo).

Fase 1 — Generación de claves (Juan)

Paso 1.1. Juan elige su clave privada, un entero aleatorio en $[1, 12]$:

$d = 8$

Paso 1.2. Calcula su clave pública mediante multiplicación escalar $Q = 8 \cdot G$. El algoritmo double-and-add convierte 8 a binario (`1000`) y, dado que solo el bit más significativo es 1, realiza tres duplicaciones consecutivas:

bit 1 $\rightarrow R = G = (1, 1)$

bit 0 $\rightarrow R = 2 \cdot R = 2G = (2, 1)$ [requiere $2^{-1} \pmod{11} = 6$]

bit 0 $\rightarrow R = 2 \cdot R = 4G = (5, 8)$ [requiere $2^{-1} \pmod{11} = 6$]

bit 0 $\rightarrow R = 2 \cdot R = 8G = (6, 7)$ [requiere $5^{-1} \bmod 11 = 9$]

$Q = 8 \cdot G = (6, 7)$

Ya en este primer paso el algoritmo de Euclides extendido interviene tres veces: cada duplicación de punto exige calcular la pendiente de la tangente, cuya fórmula contiene una división modular. El resultado se obtiene con 3 sumas de puntos en lugar de 8.

Alice publica $Q = (6, 7)$ y mantiene $d = 8$ en secreto.

Fase 2 — Firma del mensaje (Juan)

Juan desea firmar el mensaje "Ho1a".

Paso 2.1 — Hash. El mensaje se convierte en un escalar sumando los valores ASCII de sus caracteres módulo n :

"Hola" $\rightarrow 72 + 111 + 108 + 97 = 388$

$h = 388 \bmod 13 = 11$

Paso 2.2 — Nonce. Alice elige un entero aleatorio secreto, usado exclusivamente para esta firma:

$k = 5$

Paso 2.3 — Punto R. Calcula $R = k \cdot G$ mediante double-and-add:

$R = 5 \cdot G = (6, 4)$

Paso 2.4 — Valor r. Toma la coordenada x de R , reducida módulo n :

$r = 6 \bmod 13 = 6$

Paso 2.5 — Valor s. Aplica la fórmula de firma, que requiere el inverso modular de k :

$k^{-1} = 5^{-1} \bmod 13 = 8$ [Euclides: $5 \cdot 8 = 40 \equiv 1 \pmod{13}$ ✓]

$s = k^{-1} \cdot (h + d \cdot r) \bmod 13$

$= 8 \cdot (11 + 8 \cdot 6) \bmod 13$

$$= 8 \cdot 59 \bmod 13$$

$$= 4$$

Resultado. La firma es el par:

$$(r, s) = (6, 4)$$

Juan transmite el mensaje "Hola" en texto claro, acompañado del par (6, 4). Ni d ni k viajan por el canal.

Fase 3 — Verificación (Jorge)

Jorge recibe el mensaje "Hola" y la firma (6, 4). Conoce la clave pública $Q = (6, 7)$ pero **desconoce d y k** .

Paso 3.1 — Hash. Recalcula el hash del mensaje recibido:

$$h = 11$$

Paso 3.2 — Inverso de s .

$$w = s^{-1} \bmod 13 = 4^{-1} \bmod 13 = 10 \quad [\text{Euclides: } 4 \cdot 10 = 40 \equiv 1 \pmod{13} \checkmark]$$

Paso 3.3 — Pesos auxiliares.

$$u_1 = h \cdot w \bmod 13 = 11 \cdot 10 \bmod 13 = 110 \bmod 13 = 6$$

$$u_2 = r \cdot w \bmod 13 = 6 \cdot 10 \bmod 13 = 60 \bmod 13 = 8$$

Paso 3.4 — Reconstrucción del punto X . Se realizan dos multiplicaciones escalares y una suma de puntos:

$$u_1 \cdot G = 6 \cdot (1, 1) = (7, 2)$$

$$u_2 \cdot Q = 8 \cdot (6, 7) = (1, 10)$$

$$X = (7, 2) + (1, 10) = (6, 4)$$

Paso 3.5 — Comprobación.

$$X \cdot x \bmod n = 6 \bmod 13 = 6$$

$$r = 6$$

$$6 = 6 \rightarrow \text{FIRMA VÁLIDA } \checkmark$$

Es importante notar que el punto reconstruido $X = (6, 4)$ coincide exactamente con el punto $R = (6, 4)$ que Alice generó en el paso 2.3, pese a que Jorge nunca conoció k . Esta coincidencia no es casual: es la manifestación numérica de la identidad algebraica demostrada en la sección 4.4, donde $u_1 \cdot G + u_2 \cdot Q = k \cdot G = R$ gracias a la distributividad y conmutatividad del grupo abeliano.

Fase 4 — Detección de una alteración

Supongan ahora que un atacante intercepta el mensaje y lo modifica a "Hola[modificado]", conservando la firma original $(6, 4)$ con la intención de hacerla pasar por auténtica.

Paso 4.1 — El hash cambia.

$$h' = \text{hash}(\text{"Hola[modificado]"}) \bmod 13 = 12 \quad (\text{antes era } 11)$$

Paso 4.2 — Los pesos cambian. El valor w no varía (depende solo de s), pero u_1 sí:

$$u_1' = h' \cdot w \bmod 13 = 12 \cdot 10 \bmod 13 = 3 \quad (\text{antes } 6)$$

$$u_2 = r \cdot w \bmod 13 = 6 \cdot 10 \bmod 13 = 8 \quad (\text{sin cambio})$$

Paso 4.3 — El punto reconstruido cambia.

$$u_1' \cdot G = 3 \cdot (1, 1) = (8, 10) \quad (\text{antes } (7, 2))$$

$$u_2 \cdot Q = 8 \cdot (6, 7) = (1, 10)$$

$$X' = (8, 10) + (1, 10) = (2, 1) \quad (\text{antes } (6, 4))$$

Paso 4.4 — La verificación falla.

$$X' \cdot x \bmod n = 2 \bmod 13 = 2$$

$$r = 6$$

$2 \neq 6 \rightarrow$ FIRMA INVÁLIDA ✗

La alteración se detecta automáticamente. El hash del mensaje está incrustado en la ecuación de verificación, de modo que cualquier modificación —por mínima que sea— desplaza el punto reconstruido a una posición distinta del grupo, rompiendo la igualdad. El atacante no puede corregir la firma sin conocer la clave privada d .

Resumen de valores del ejemplo

Fase	Elemento	Valor
Configuración	Curva / G / n	$y^2 \equiv x^3 + 4x + 7 \pmod{11} / (1,1) / 13$
Claves	d (privada)	8
Claves	$Q = d \cdot G$ (pública)	(6, 7)
Firma	$h = \text{hash}(\text{"Hola"})$	11
Firma	k (nonce)	5
Firma	$R = k \cdot G$	(6, 4)
Firma	$r = R.x \pmod{n}$	6
Firma	$k^{-1} \pmod{13}$	8
Firma	s	4
Firma	Firma (r, s)	(6, 4)
Verificación	$w = s^{-1} \pmod{13}$	10
Verificación	u_1 / u_2	6 / 8
Verificación	$X = u_1 \cdot G + u_2 \cdot Q$	(6, 4) = R ✓
Verificación	Resultado	VÁLIDA
Alteración	h'	12

Fase	Elemento	Valor
Alteración	X'	(2, 1)
Alteración	Resultado	INVÁLIDA

Dónde interviene cada concepto del curso

El recorrido anterior permite localizar con precisión cada tema de la asignatura:

- **Euclides extendido** aparece 3 veces en la generación de claves (una por duplicación), varias veces dentro de cada multiplicación escalar de la firma y la verificación, una vez para k^{-1} en el paso 2.5 y una vez para s^{-1} en el paso 3.2.
- **La suma de puntos** es la operación base: se ejecuta dentro de cada double-and-add y explícitamente en el paso 3.4 al combinar $u_1 \cdot G$ con $u_2 \cdot Q$.
- **El grupo cíclico** garantiza que todos los puntos intermedios —(2,1), (5,8), (6,7), (6,4), (7,2), (1,10), (8,10), (2,1)— pertenezcan al conjunto de 13 elementos, sin excepción (axioma de cierre).
- **La estructura abeliana** es lo que hace que X reconstruya exactamente R en el paso 3.4.

6. Caso de estudio de seguridad: ataque por reutilización de nonce

El nonce k debe ser único e impredecible en cada firma. Si se reutiliza el mismo k para firmar dos mensajes distintos, la clave privada queda completamente expuesta usando únicamente aritmética modular, sin necesidad de resolver el ECDLP.

6.1 Fundamento del ataque

Con el mismo k , ambas firmas comparten el mismo r (pues $R = k \cdot G$ es idéntico). Entonces:

$$s_1 = k^{-1}(h_1 + d \cdot r) \bmod n$$

$$s_2 = k^{-1}(h_2 + d \cdot r) \bmod n$$

$$\text{Restando: } s_1 - s_2 = k^{-1}(h_1 - h_2) \bmod n$$

$$\text{Despejando: } k = (h_1 - h_2) \cdot (s_1 - s_2)^{-1} \bmod n$$

Una vez conocido k, se despeja la clave privada:

$$d = (s_1 \cdot k - h_1) \cdot r^{-1} \bmod n$$

5.2 Implementación

```
def ataque_reutilizacion_nonce(mensaje1, firma1, mensaje2, firma2, curva=None):
    n = curva['n']
    r1, s1 = firma1
    r2, s2 = firma2

    if r1 != r2:
        return None # el nonce no fue reutilizado

    r = r1
    h1 = hash_mensaje(mensaje1, n)
    h2 = hash_mensaje(mensaje2, n)

    # Recuperar k
    dif_s_inv = inverso_modular((s1 - s2) % n, n)
    k = ((h1 - h2) * dif_s_inv) % n

    # Recuperar d
    r_inv = inverso_modular(r, n)
    d = ((s1 * k - h1) * r_inv) % n

    return k, d
```

Este caso ilustra de forma contundente que la seguridad de ECDSA no depende únicamente de la solidez matemática del ECDLP, sino también de una implementación correcta. Un incidente real de este tipo afectó la consola PlayStation 3 en 2010, cuando la reutilización de un nonce permitió extraer la clave de firma del sistema.

7. Elección de parámetros: la importancia del orden primo

Durante el desarrollo se detectó que la curva $y^2 \equiv x^3 + 4x + 7$ sobre Z_{23} genera un grupo de orden 24, que no es primo. Un orden compuesto provoca que ciertos escalares carezcan de inverso modular en las operaciones del protocolo, lo que rompe ECDSA y además introduce vulnerabilidades (el ataque de Pohlig-Hellman aprovecha la factorización del orden). Por ello se optó por trabajar sobre Z_{11} , donde el mismo polinomio produce un grupo de orden 13, primo. Con orden primo se garantiza que todo elemento distinto de O sea generador y que todos los inversos requeridos existan.

Esta decisión conecta directamente con un resultado del curso: en un grupo cíclico de orden primo, todo elemento no neutro genera el grupo completo.

8. Conexión con los temas del curso

Tema del curso	Aplicación en el proyecto
Grupos y axiomas	El conjunto $E(Z_{11})$ verifica cierre, asociatividad, neutro, inverso
Grupos cíclicos y generadores	$G = (1,1)$ genera los 13 elementos del grupo
Orden de un grupo/elemento	$n = 13$ es el orden de G ; debe ser primo
Isomorfismo de grupos	ECDSA y DSA comparten estructura; $\varphi(k) = k \cdot G$ relaciona Z_{13} con $E(Z_{11})$
Aritmética modular	Todas las operaciones de firma y verificación son en Z_n
Algoritmo de Euclides extendido	Cálculo de inversos modulares en cada fase del protocolo

9. Conclusiones

El proyecto demuestra que un primitivo criptográfico de uso real puede construirse a partir de los conceptos fundamentales de un curso de matemáticas discretas. La teoría de grupos no es un marco decorativo, sino la razón por la que el protocolo funciona: la correctitud de la verificación se sigue directamente de la asociatividad y conmutatividad del grupo abeliano de puntos, y la seguridad se apoya en la dificultad del logaritmo discreto en ese grupo. El algoritmo de Euclides extendido, por su parte, actúa como el puente aritmético que hace posible cada operación, apareciendo en la suma de puntos, en la firma y en la verificación.

La implementación desde cero, evitando librerías criptográficas, permitió comprender el porqué de cada paso en lugar de tratarlo como una caja negra. El caso del ataque por reutilización de nonce refuerza una lección central de la criptografía aplicada: la robustez teórica de un algoritmo puede anularse por completo con un solo error de implementación.

Como líneas de trabajo futuro, el sistema podría extenderse a curvas estándar de tamaño realista (como secp256k1), incorporar una función hash criptográfica real (SHA-256) en lugar de la versión simplificada usada con fines didácticos, y añadir el protocolo de intercambio de claves ECDH sobre la misma base algebraica.

Referencias

- Hankerson, D., Menezes, A., & Vanstone, S. (2004). *Guide to Elliptic Curve Cryptography*. Springer.
- Washington, L. C. (2008). *Elliptic Curves: Number Theory and Cryptography*. Chapman & Hall/CRC.
- Standards for Efficient Cryptography Group (SECG). *SEC 1: Elliptic Curve Cryptography*.
- Johnson, D., Menezes, A., & Vanstone, S. (2001). The Elliptic Curve Digital Signature Algorithm (ECDSA). *International Journal of Information Security*.